



# Introducing JSON

ECMA-404 The JSON Data Interchange Standard.

**JSON** (JavaScript Object Notation) is a lightweight data-interchange format. It is easy for humans to read and write. It is easy for machines to parse and generate. It is based on a subset of the JavaScript Programming Language Standard ECMA-262 3rd Edition - December 1999. JSON is a text format that is completely language independent but uses conventions that are familiar to programmers of the C-family of languages, including C, C++, C#, Java, JavaScript, Perl, Python, and many others. These properties make JSON an ideal data-interchange language.

JSON is built on two structures:

A collection of name/value pairs. In various languages, this is realized as an *object*, record, struct, dictionary, hash table, keyed list, or associative array.  
An ordered list of values. In most languages, this is realized as an *array*, vector, list, or sequence.

These are universal data structures. Virtually all modern programming languages support them in one form or another. It makes sense that a data format that is interchangeable with programming languages also be based on these structures.

In JSON, they take on these forms:

An *object* is an unordered set of name/value pairs. An object begins with { *left brace* and ends with } *right brace*. Each name is followed by : *colon* and the name/value pairs are separated by , *comma*.

```
json
  element

value
  object
  array
  string
  number
  "true"
  "false"
  "null"

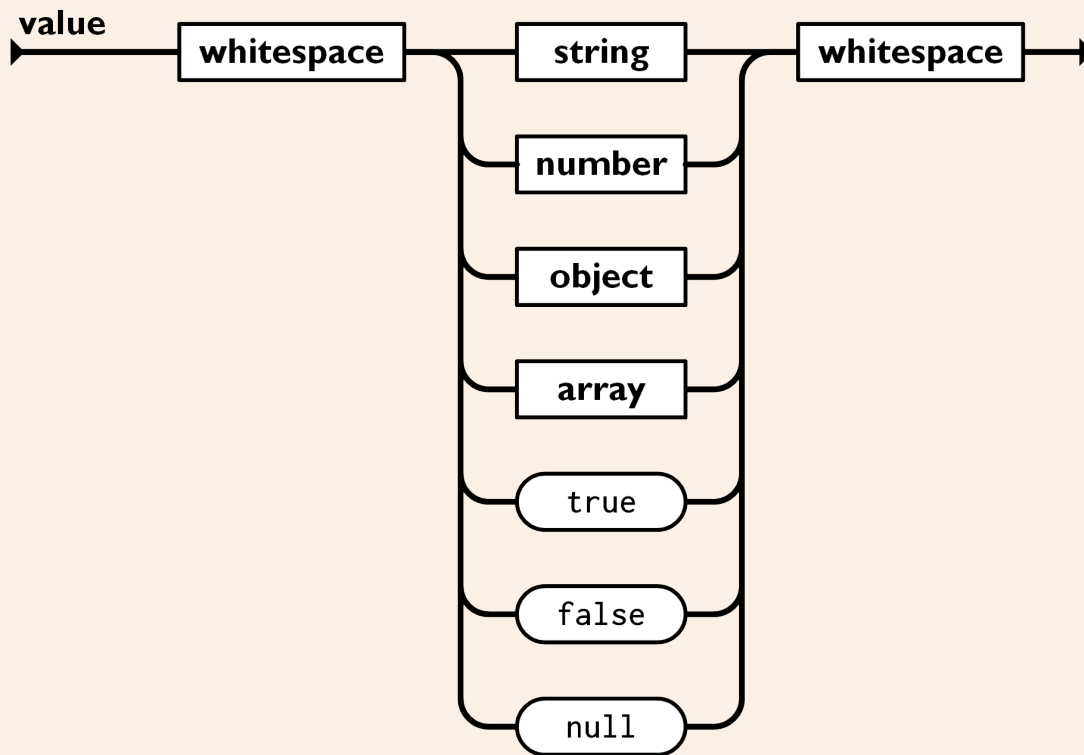
object
  '{' ws '}'
  '{' members '}'

members
  member
  member ' , ' members

member
  ws string ws ':' element

array
  '[' ws ']'
  '[' elements ']'
```





A *string* is a sequence of zero or more Unicode characters, wrapped in double quotes, using backslash escapes. A character is represented as a single character string. A string is very much like a C or Java string.

digits  
digit  
digit digits

digit  
'0'  
onenine

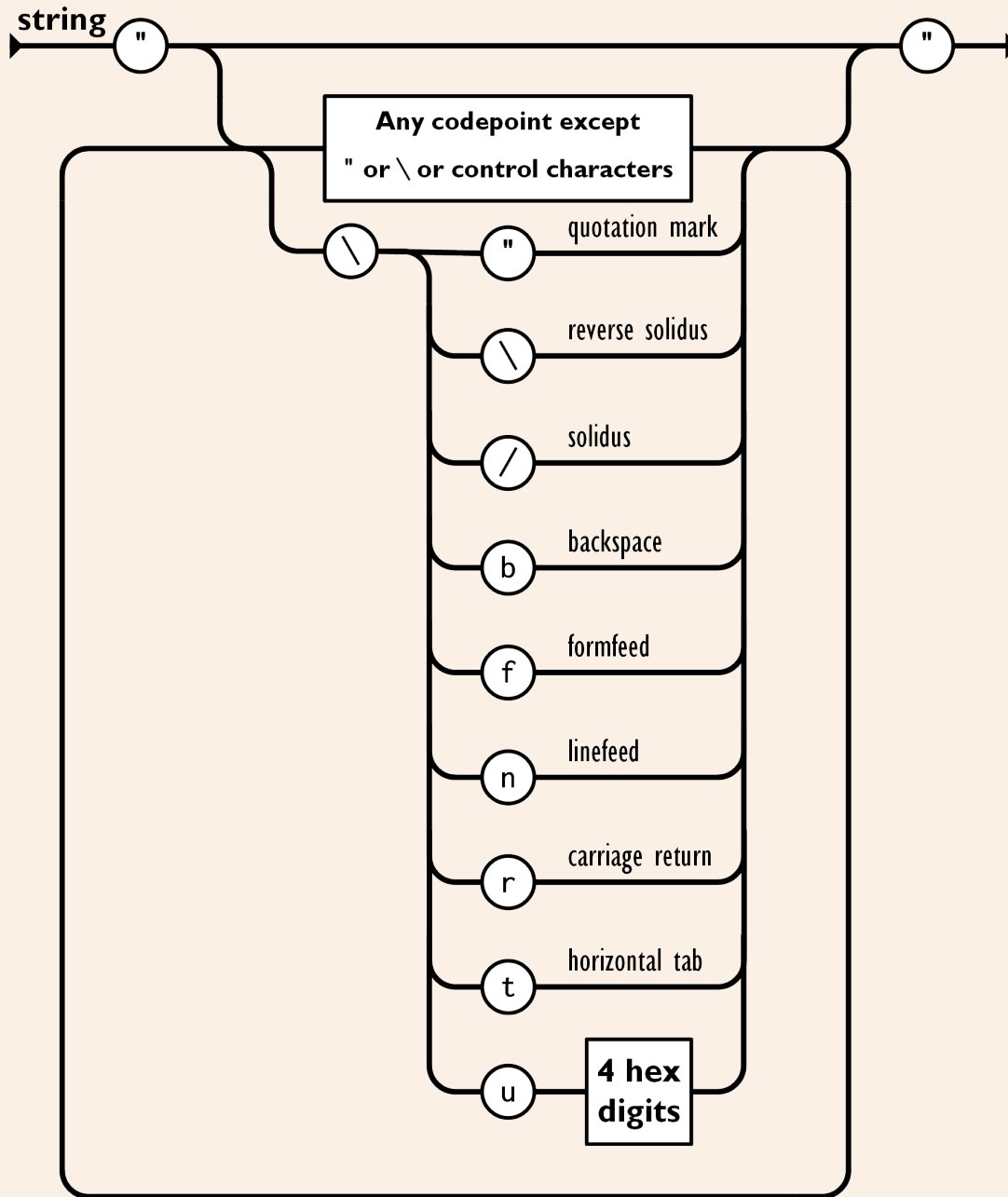
onenine  
'1' . '9'

fraction  
""  
'.' digits

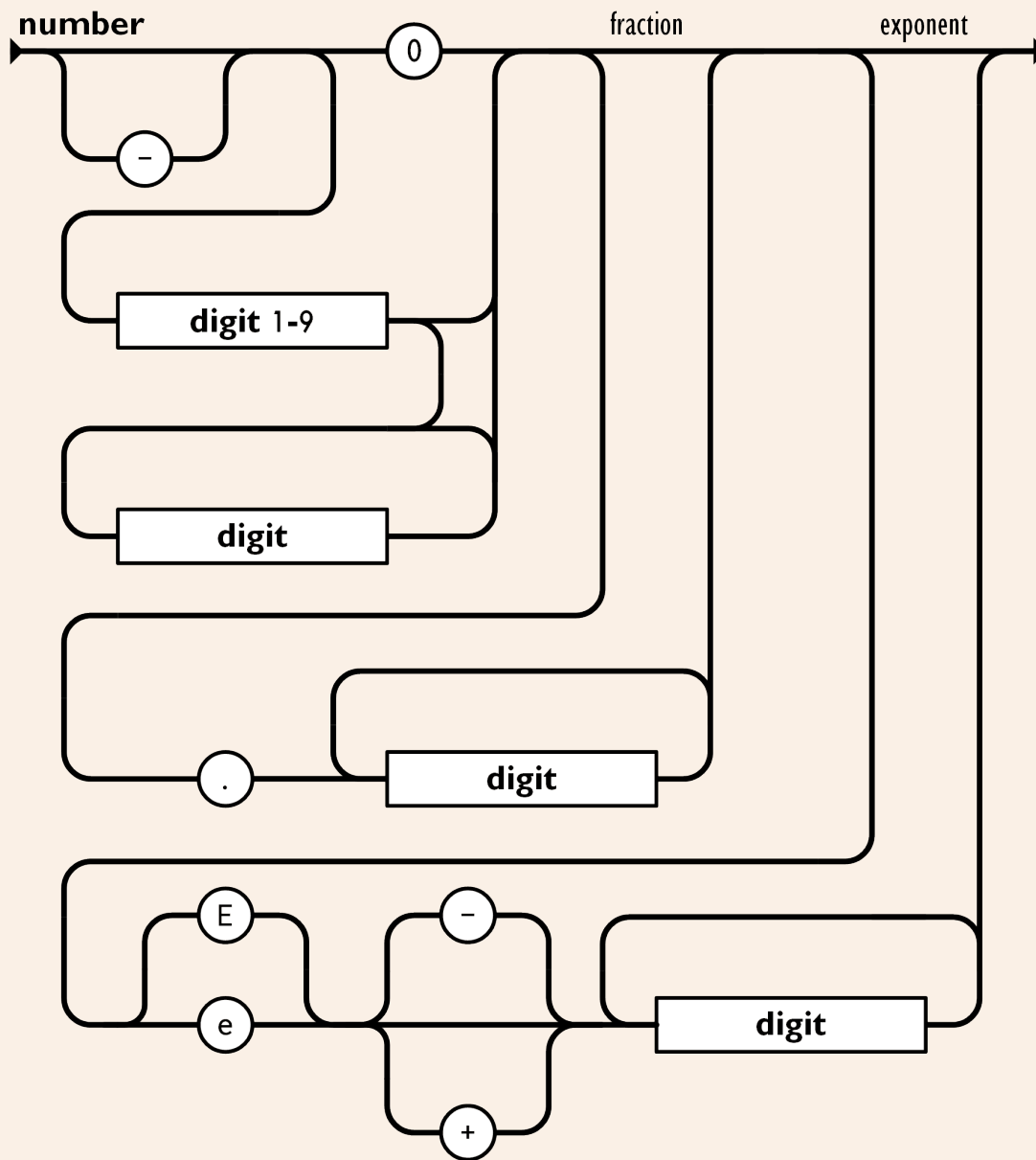
exponent  
""  
'E' sign digits  
'e' sign digits

sign  
""  
'+'  
'-'

ws  
""  
'0020' ws  
'000A' ws  
'000D' ws  
'0009' ws



A *number* is very much like a C or Java number, except that the octal and hexadecimal formats are not used.



Whitespace can be inserted between any pair of tokens. Excepting a few encoding details, that completely describes the language.

**whitespace**

